

UNIR

Universidad Internacional de La Rioja

Actividad 3

Autodiagnóstico del ascensor inteligente ACME

Instrumentación avanzada sobre la base Act1+Act2 (ESP32)

Instrumentación Electrónica

Máster Universitario en Ingeniería de Telecomunicación

Autor:

Stefan Lica Cristian Ionita

Profesores:

Ivan Araquistain

Carlota Campomanes Prieto

1. Introducción

La Actividad 3 cierra el ciclo de desarrollo del producto inteligente de ACME S.A.: sobre el sistema embebido construido en las dos actividades previas se incorpora una funcionalidad de **instrumentación avanzada** que perfecciona su eficiencia, fiabilidad y trazabilidad, alineándola con los objetivos de digitalización industrial que persigue la empresa.

El **punto de partida** es una base limpia que fusiona en un único *sketch* sobre ESP32 los dos proyectos anteriores: el **clasificador de cajas** de la Actividad 1 (HC-SR04, servo-compuerta, indicadores) y el **ascensor inteligente de cinco plantas** de la Actividad 2 (mando IR NEC con la librería *Arduino-IRremote* [1], botones, PIR, lazo ON-OFF de clima). La integración va más allá de la mera convivencia en el mismo `loop()`: cuando el clasificador confirma una caja *alta*, su altura se mapea a una planta del ascensor y se invoca `elevator_request()` automáticamente, de modo que la caja desviada al carril de ascensor en Act1 *sube físicamente* por el ascensor de Act2. La arquitectura modular de ambos proyectos — pines disjuntos y APIs claras — permitió esa fusión con cambios mínimos y sin tocar el cableado de los sensores compartidos.

Entre las tres líneas que propone el enunciado para perfeccionar el sistema —control remoto, autodiagnóstico y supervisión inteligente— se ha seleccionado el **autodiagnóstico**. Es la opción que mejor encaja con el reparto de evaluación (el Criterio 2, que vale el 40 %, premia la *ampliación del circuito E/S*) y la que aporta más valor sobre la base ya construida: introduce **sensores redundantes**, **detección de averías por límites de rango**, **promediado de medidas** y **vigilancia del consumo de actuadores**, todo ello con una **reacción escalonada** que reproduce el comportamiento de un sistema de instrumentación industrial moderno (avisar cuando hay reserva, dar alarma cuando el fallo es crítico). La nueva capa se implementa en un único módulo —*diagnostics*— que se interpone entre los sensores y el lazo de control sin alterar los módulos heredados, cumpliendo así el principio de *mínima programación* (Criterio 3).

El resto de la memoria describe el hardware fusionado (§2), la arquitectura del firmware (§3), el núcleo de la nueva capa de instrumentación (§4), la interfaz HMI y el logger ampliado (§5), las pruebas reales en Wokwi (§6) y las conclusiones (§7).

2. Hardware

El montaje fusiona en un único ESP32 DevKit C v4 los componentes de Act1 y Act2 y añade los dos elementos nuevos exigidos por el autodiagnóstico (tabla 1). El cableado completo se describe en `diagram.json` y se reproduce en la figura 1.

Componente	Función	Pines ESP32
<i>Sensores compartidos / heredados</i>		
DHT22 principal	Temperatura y humedad (sensor primario).	4
LDR	Iluminación (ADC1, <i>input-only</i>).	34
HC-SR04	Altura por ultrasonidos (clasificador).	5 (TRIG), 18 (ECHO)
LCD 20×4	HMI por I ² C (ox27).	21 (SDA), 22 (SCL)
RTC DS1307	Reloj para <i>timestamps</i> (I ² C ox68).	21 (SDA), 22 (SCL)
PIR HC-SR501	Presencia en cabina.	23
Receptor IR TSOP	Mando a distancia NEC.	15
Botones PO – P3	Llamada por planta (P4 sólo por IR; ver abajo).	32, 33, 35, 16
Botón RESET	Reset de contadores del clasificador.	14
Servo cabina	Movimiento entre plantas (PWM LEDC).	19
Servo compuerta	Desvío de cajas (PWM LEDC).	13
LED rojo/azul/amar.	Calentador, enfriador, lámpara PWM.	2, 12, 0
LED verde/rojo	Ascensor / Escalera (clasificación).	25, 26
Buzzer pasivo	Pitido de clasificación y alarma.	27
<i>Nuevos para el autodiagnóstico (Act3)</i>		
DHT22 reserva	Sensor T/H redundante (failover automático).	17
Potenciómetro	Sensor de corriente de actuadores (simulado).	39 (V _N)

Tabla 1: Componentes y asignación de pines del sistema integrado.

[Pendiente: captura de pantalla del montaje Wokwi]

Figura 1: Montaje completo en Wokwi: clasificador, ascensor, HMI y la pareja de sensores nuevos del autodiagnóstico (DHT22 de reserva y potenciómetro).

2.1. Pines nuevos del autodiagnóstico

DHT22 de reserva — GPIO17. Es la *redundancia* de temperatura y humedad pedida por la sección de autodiagnóstico del enunciado. GPIO 17 se ha liberado retirando el pulsador físico de la planta 4, ya que era el único GPIO digital bidireccional disponible tras Act1 y Act2: el ESP32 sólo deja libres GPIO 36 y 39, ambos *input-only*, no aptos para el bus bidireccional del DHT22. **La planta 4 sigue siendo plenamente accesible** mediante el botón 4 del mando IR; los pulsadores físicos y el mando IR fueron siempre redundantes entre sí, así que la operativa del ascensor no pierde ninguna planta.

Sensor de corriente — GPIO39 (V_N). Un potenciómetro modela la *salida ya acondicionada* de un sensor de corriente real (resistencia *shunt* más amplificador): en Wokwi no se puede variar el consu-

mo real de un LED, así que el potenciómetro actúa como inyector de fallo de *sobreconsumo* para la demostración, y al mismo tiempo deja en el diagrama el divisor analógico que tendría su contrapartida hardware en un montaje físico. GPIO 39 (etiquetado V_N en la DevKit) es *input-only* y pertenece a ADC1, evitando cualquier conflicto con el LDR (GPIO 34) o con Wi-Fi (que reservaría ADC2). El escalado lineal es 0 mA a 800 mA con umbral de sobreconsumo en 500 mA.

Los pines de Act1 y Act2 se eligieron con la regla de *conjuntos disjuntos* salvo los sensores compartidos, lo que permitió la fusión sin cambios de cableado. Los *strapping pins* del ESP32 [2] mantienen niveles correctos al arranque: los LEDs de control parten apagados y el receptor IR queda en HIGH, compatible con el reposo del TSOP.

3. Firmware

El firmware está organizado en doce módulos C++ con responsabilidades separadas (tabla 2). Once de ellos heredan, sin cambios o con adaptaciones mínimas, las responsabilidades de Act1 y Act2; el doceavo —*diagnostics*— es el módulo nuevo de esta entrega. El `.ino` actúa como orquestador: invoca a cada módulo en el orden adecuado pero no habla directamente con el hardware, manteniendo el patrón modular que ya documentaron las dos memorias anteriores.

Módulo	Responsabilidad
<code>config.h</code>	Pines, <i>setpoints</i> , umbrales y constantes del autodiagnóstico.
<code>sensors.{h,cpp}</code>	HC-SRO4, DHT22 principal y reserva , LDR, PIR, botones y sensor de corriente .
<code>actuators.{h,cpp}</code>	Dos servos (cabina y compuerta), LEDs de control y de clasificación, buzzer.
<code>classifier.{h,cpp}</code>	FSM del clasificador + puente al ascensor (altura → planta).
<code>elevator.{h,cpp}</code>	FSM del ascensor y cola FIFO.
<code>control.{h,cpp}</code>	Lazos ON-OFF con histéresis y modelo térmico simulado.
<code>ircontrol.{h,cpp}</code>	Decodificación del mando IR (NEC).
<code>display.{h,cpp}</code>	HMI sobre LCD 20×4 con cuatro líneas simultáneas.
<code>logger.{h,cpp}</code>	Telemetría CSV unificada (clasif. + ascensor + clima + diagnóstico).
<code>diagnostics.{h,cpp}</code>	Capa nueva (Act3) : validación, conmutación, promediado y alarma.
<code>actividad_3_vs.ino</code>	<code>setup()</code> y <code>loop()</code> no bloqueantes.

Tabla 2: Módulos del firmware integrado.

3.1. Cadena del `loop()`

Igual que en las actividades anteriores, el bucle principal es cooperativo y no bloqueante: cada subsistema se autorregula con `millis()`. La diferencia respecto a Act2 es que las lecturas crudas de los sensores ambientales ya no se hacen en el `.ino`: ahora pasan por `diag_tick()`, que las valida, las promedia, decide la fuente activa y entrega al control únicamente medidas *ya validadas*.

```

1 void loop() {
2     int8_t pIR = ircontrol_poll(); if (pIR >= 0) elevator_request(pIR);
3     int8_t pBT = buttonsPoll();    if (pBT >= 0) elevator_request(pBT);
4     classifier_tick();             // FSM + puente al ascensor si caja ALTA
5     elevator_tick();
6     diag_tick();                  // lee, valida, conmuta a reserva, promedia,
                                   alarma
7     if (diag_hasClimate())
8         control_tick(diag_temp(), diag_hum(), diag_lux());
9     displayUpdate(); logger_tick();
10 }

```

Listing 1: Cadena del `loop()` integrado.

3.2. Integración Act1 ↔ Act2

El módulo `classifier` encapsula la FSM heredada de Act1. Al confirmar una caja con altura inferior a `UMBRAL_ALTA_CM` ejecuta el **puente al ascensor**: mapea la altura al rango `[0, NUM_PLANTAS-1]` con `map()` (caja más alta → planta más alta) y llama a `elevator_request(planta)`. Así el carril ASCENSOR de Act1 deja de ser un terminador estático y se convierte en la entrada real al ascensor de Act2.

4. Autodiagnóstico

El módulo `diagnostics` es la **capa de instrumentación avanzada** que se interpone entre las lecturas crudas de los sensores y el lazo de control de clima. En cada `diag_tick()` ejecuta cinco acciones encadenadas: lee los sensores monitorizados, valida cada lectura por rango, mantiene una media móvil de las medidas válidas, decide la fuente activa de temperatura y de humedad por separado, y reacciona ante el fallo con una severidad escalonada que puede llegar a inhibir el ascensor. El diagrama de bloques se muestra en la figura 2.

[Pendiente: diagrama de bloques del módulo *diagnostics*]

Figura 2: Tubería de `diagnostics`: lectura → validación por rango → conmutación independiente T/H → promediado → severidad.

Detección de fallos por límites de rango. A cada sensor monitorizado se le asocia una *banda de operación plausible* en su unidad física (tabla 3). Una lectura fuera de la banda —o un NaN en el caso del DHT22, que la librería devuelve ante fallo del protocolo— se interpreta como **cortocircuito o circuito abierto** de la conexión del sensor. Para la temperatura y la humedad, los límites se han elegido más estrechos que el rango total del chip ($-40..80^{\circ}\text{C}$ y $0..100\%$ según [3]): así un valor físicamente posible pero implausible para el entorno de la planta se trata también como avería. Para evitar

disparos por ruido, se exige que `DIAG_FAULT_SAMPLES` (3) lecturas consecutivas sean malas antes de declarar la avería; igualmente se exigen 3 buenas seguidas antes de dar el sensor por recuperado (anti-rebote tipo Schmitt sobre la salud del sensor).

Magnitud	Banda válida	Sensor	Justificación
Temperatura	-10°C a 50°C	DHT22 (pri/res)	Banda operativa industrial (más estrecha que el rango total del sensor, $-40..80^{\circ}\text{C}$).
Humedad	15 % a 100 %	DHT22 (pri/res)	$< 15\%$ implausible en aire ambiente.
Iluminación	1 lux a 20 000 lux	LDR	Interior industrial; $> 20\,000$ lux son exteriores.
Corriente	$\leq 500\text{ mA}$	Pot. acond.	Umbral de <i>sobreconsumo</i> (§4.1).

Tabla 3: Bandas de operación plausible y su justificación.

Promediado de medidas (media móvil). Para reducir el ruido de medida [4] todas las lecturas válidas pasan por una **media móvil** de `DIAG_AVG_WINDOW=5` muestras antes de alimentar al control y al *display*. El búfer de cada magnitud es independiente y descarta las muestras marcadas como malas, de modo que un fallo transitorio no contamina la media. Cuando una magnitud conmuta de sensor (ver §4.1), su búfer se **vacía** para no mezclar lecturas de dos físicas distintas.

4.1. Redundancia independiente de temperatura y humedad

El enunciado pide *duplicar el sensor de temperatura para que, si uno falla, se active el sensor de reserva*. En esta entrega se ha llevado un paso más allá: el DHT22 tiene un elemento de medida *independiente* para temperatura y otro para humedad, por lo que en la práctica puede degradarse una magnitud sin la otra. El módulo lo refleja con cuatro monitores de salud independientes (`monPriT`, `monBckT`, `monPriH`, `monBckH`) y dos indicadores de fuente activa (`usingBackupT` y `usingBackupH`). El resultado es que el sistema puede estar usando el DHT22 *de reserva* para la temperatura y al mismo tiempo el *principal* para la humedad, o viceversa. La conmutación es automática y reversible: cuando un sensor en fallo se recupera, su monitor confirma 3 lecturas buenas seguidas y la fuente vuelve al principal.

El proceso de selección es: si el principal está sano se usa siempre el principal; si no, se usa la reserva siempre que esté sana; si ambos fallan se mantiene la última fuente activa. Cuando la fuente cambia, se vacía el búfer de la media móvil para no mezclar lecturas de sensores distintos.

Diagnóstico de actuadores por sensor de corriente. La parte de actuadores se cubre con un **sensor de corriente** en GPIO 39 (V_N). En un montaje real correspondería a la salida ya acondicionada de una resistencia *shunt* amplificada; en Wokwi se modela con un potenciómetro, pues no es posible variar dinámicamente el consumo real de un LED. El ADC se promedia con la misma ventana móvil que el resto de magnitudes —imprescindible para no disparar falsas alarmas con el ruido del ADC del

ESP32— y se compara con `DIAG_CURRENT_MAX_MA=500 mA`: si la corriente promediada supera ese umbral, se declara **sobreconsumo** y la severidad sube a ALARMA.

4.2. Reacción escalonada: OK → AVISO → ALARMA

La decisión de severidad combina los siete monitores de salud anteriores con una jerarquía sencilla pero realista (tabla 4). La idea es distinguir un fallo con respaldo (donde el sistema puede seguir operando) de un fallo crítico que exige detener el ascensor.

Severidad	Condición	Reacción
OK	Todos los sensores sanos y operando con el principal.	Operación normal.
AVISO	Algún DHT principal falló pero hay reserva sana (sólo en T, sólo en H, o en ambas).	Conmuta a reserva, lo indica en LCD y CSV, el sistema sigue operando; <i>sin alarma sonora</i> .
ALARMA	Fallo crítico sin reserva: ambos DHT en T, o ambos en H, o LDR averiado, o sobreconsumo.	El ascensor pasa al estado ALARM (cabinas congeladas), el buzzer pita cada 1 s y el LCD muestra el banner del fallo. Al desaparecer el fallo, el sistema vuelve a IDLE automáticamente.

Tabla 4: Severidad escalonada del autodiagnóstico.

Cuando la severidad es ALARMA, `diagnostics` llama a `elevator_setAlarm(true)` —una pequeña API que se ha añadido al módulo `elevator` para que el autodiagnóstico pueda forzar y liberar el estado ALARM de la FSM existente. La cola de plantas se conserva durante la alarma, de modo que al despejarse el fallo el ascensor reanuda los destinos pendientes sin que el operador deba reintroducirlos.

5. HMI y logger

5.1. Pantalla LCD 20×4

El LCD 16×2 de Act2 se ha sustituido por uno de **20 columnas × 4 filas** (`wokwi-lcd2004`, misma librería `LiquidCrystal_I2C` en 0x27). Con cuatro líneas ya no hace falta rotar variables: el HMI muestra las tres funciones del sistema simultáneamente y reserva la línea superior para la información crítica del autodiagnóstico:

Línea	Contenido
Lo codeinCAJA#07 ASCENSOR P3, o el banner del autodiagnóstico si la severidad no es OK (p. ej. codeinAVISO H->RESERVA o !ALARMA SENSOR LUZ).	Última caja clasificada, p. ej.
L1 codeinELEV MOVING Po>P3.	Estado del ascensor, p. ej.
L2	Clima: T23.0P H55 %R L el marcador de fuente a R = sensor de reserva. Lo conmutación de la hume
L3	Contadores y diagnóstico jas a ASCENSOR y a ESCA presencia PIR).

5.2. Logger CSV unificado

El *logger* emite una fila de telemetría cada LOG_PERIOD_MS (1 s) al puerto serie a 115 200 Bd, con la cabecera impresa una sola vez al arrancar. Mantiene el formato CSV plano de Act2 y añade siete columnas de diagnóstico que documentan la nueva capa de instrumentación (tabla 5). El *timestamp* es absoluto si el RTC DS1307 responde (YYYY-MM-DD HH:MM:SS); si no, se usa *millis()* en formato relativo.

Bloque	Columnas
Marca de tiempo	timestamp.
Clasificación	piece_n, box_h_cm, box_class.
Ascensor	floor_cur, floor_tgt, elev_state.
Clima (control)	T_C, H_pct, Lux_lx — valores validados y promediados que recibe el control.
Actuadores	heater, cooler, lamp_pwm, humidifier, dehumidifier, pir.
Autodiagnóstico	temp_src, hum_src (P/R independientes); t_pri_ok, t_bck_ok, h_pri_ok, h_bck_ok (salud de cada elemento, 1/O); ldr_ok, act_ok; current_ma (corriente promediada del bloque de actuadores); diag_sev (OK/AVISO/ALARMA).
Lecturas crudas	t_raw, h_raw, lux_raw — lo que entregó cada sensor antes de validar y promediar; deja <i>evidencia</i> del valor que provocó cualquier avería.

Tabla 5: Bloques de columnas del CSV unificado.

La distinción entre las columnas *validadas* (T_C, ...) y las *crudas* (t_raw, ...) es la clave para auditar el funcionamiento del autodiagnóstico: durante un fallo, las primeras se *congelan* en el último valor válido (el control no debe ver datos sospechosos), mientras que las segundas muestran la lectura real que llevó a declarar la avería.

6. Resultados

Las pruebas se han realizado sobre el simulador Wokwi [5] integrado con VS Code. El circuito está descrito en `diagram.json` y se arranca con `F1` → *Wokwi: Start Simulator* tras compilar con `arduino-cli -fqbn esp32:esp32:esp32` .. Los valores ambientales se controlan con los *sliders* de los componentes `wokwi-dht22` (principal y reserva), `wokwi-photoresistor` y del potenciómetro de corriente. Las llamadas al ascensor se hacen con los botones físicos PO-P3, el mando IR (incluido el botón 4 que cubre la **planta 4**) y la propia entrega de cajas altas desde el clasificador.

Sesión representativa. La traza siguiente reproduce, simplificada, una sesión real del monitor serie a 115 200 Bd. Se ha bajado el slider del DHT22 principal por debajo de -10°C , después se ha bajado la humedad del principal al 12 % y por último se ha cranado el slider del LDR por encima de 20 000 lux; en cada paso el sistema diagnostica el fallo y reacciona.

```

1 22:58:00,...,23.00,55.00,400.26,...,PRI,PRI,1,1,1,1,1,1,...,OK,    23.00, 55.00,  400.26
2 -- principal T -> -13,7 C (fuera de -10..50); tras 3 ciclos del DHT:
3 [DIAG] fuente de TEMPERATURA -> RESERVA
4 22:58:27,...,23.00,67.60,6928.50,...,BCK,PRI,0,1,1,1,1,1,...,AVISO,-13.70, 76.00, 6928.50
5 -- principal T -> 6,5 C (en rango), recupera:
6 [DIAG] fuente de TEMPERATURA -> PRINCIPAL
7 22:59:32,...,6.50, 76.00,6928.50,...,PRI,PRI,1,1,1,1,1,1,...,OK,    6.50, 76.00, 6928.50
8 -- principal H -> 12 % (fuera de 15..100); conmuta SOLO humedad:
9 [DIAG] fuente de HUMEDAD -> RESERVA
10 23:00:21,...,6.50,87.50,6928.50,...,PRI,BCK,1,1,0,1,1,1,...,AVISO, 6.50, 12.00, 6928.50
11 -- LDR -> >20000 lux; sin reserva -> ALARMA:
12 23:00:35,...,3.54,87.50,6928.50,...,PRI,BCK,1,1,0,1,0,1,...,ALARMA,-8.30,12.00,27795.06
13 -- LDR vuelve a rango; alarma se libera, AVISO por humedad continua:
14 23:00:41,...,0.58,87.50,2880.37,...,PRI,BCK,1,1,0,1,1,1,...,AVISO,-8.30, 12.00, 2880.37

```

Listing 2: Traza CSV (extracto). Cabecera: `ts,..., T_C,H_pct,Lux_lx,...,temp_src,hum_src, t_pri_ok,t_bck_ok,h_pri_ok,h_bck_ok, ldr_ok,act_ok,...,diag_sev, t_raw,h_raw,lux_raw`.

Casos verificados.

- **Conmutación y recuperación de temperatura.** A las 22:58:27, tras 3 ciclos de DHT con `t_raw=-13,70` (≈ 6 s de anti-rebote), `t_pri_ok` pasa a 0, la fuente cambia a BCK y la severidad sube a AVISO; al devolver el slider a $6,5^{\circ}\text{C}$ la fuente regresa sola a PRI y baja a OK. La humedad permanece en PRI todo el rato.
- **Conmutación de humedad independiente.** A las 23:00:21, `h_raw=12 % < 15 %` provoca que sólo la humedad pase a la reserva (`hum_src=BCK, h_pri_ok=0`) mientras `temp_src` sigue en PRI: demostración cruzada de la independencia T/H.
- **Alarma crítica de LDR y recuperación.** Sin reserva en iluminación, `lux_raw=27795 > 20 000 lux` dispara ALARMA: el ascensor pasa a ALARM, suena el buzzer y el LCD muestra **!ALARMA SENSOR LUZ**. Al devolver el slider, la alarma se libera y el ascensor vuelve a IDLE sin perder la cola.
- **Promediado y datos crudos.** Las columnas validadas (`T_C, Lux_lx`) varían suavemente respecto

a las crudas (`t_raw`, `lux_raw`) y se *congelan* si la cruda queda fuera de rango: el control nunca actúa sobre un dato sospechoso, pero la cruda deja constancia en el CSV.

- **Sobreconsumo de actuadores.** En pruebas adicionales, subir el potenciómetro por encima de 500 mA pone `act_ok=0` y dispara ALARMA; el promediado de la corriente elimina los disparos por ruido del ADC.
- **Integración y planta 4 por IR.** El clasificador sigue diviendo cajas; una caja alta (`[CLF] caja #6 ALTA 10 cm`) dispara `elevator_request(2)` automáticamente. El botón 4 del mando IR sigue solicitando la planta 4 pese a no tener pulsador físico.

7. Conclusiones

Se han cumplido los objetivos de la Actividad 3 y se ha cerrado el ciclo de desarrollo del producto inteligente de ACME S.A.. La base fusionada Act1+Act2 incorpora ahora una capa de **instrumentación avanzada** que aporta cuatro funciones de autodiagnóstico —redundancia con conmutación automática, detección de averías por límites de rango, promediado de medidas y vigilancia de la corriente de actuadores— y reacciona ante el fallo con una jerarquía escalonada OK/AVISO/ALARMA que distingue entre situaciones con respaldo y fallos críticos. La diagnosis de temperatura y humedad es *independiente*, reflejando que el DHT22 tiene elementos de medida distintos para cada magnitud.

La solución encaja con los cuatro criterios de evaluación: (C1) se han probado y documentado las partes del ejemplo de autodiagnóstico, con trazas CSV reales del sistema en sus distintos modos de fallo; (C2) el circuito E/S se ha ampliado con dos componentes nuevos —un DHT22 de reserva y un sensor de corriente acondicionado— conectados a los únicos pines que quedaban libres en el ESP32 tras Act1+Act2; (C3) el funcionamiento es eficiente y la programación mínima: *toda* la capa nueva vive en un único módulo `diagnostics` y el resto de módulos heredados se han tocado lo justo (sólo un *getter* adicional en `elevator` para forzar el estado ALARM); (C4) la presentación sigue el formato pedido (Carlito $\equiv$ Calibri 12, interlineado 1.5) y respeta las diez páginas.

Como nota explícita: la **planta 4** se ha quedado sin pulsador físico para que el GPIO 17 acoja el segundo DHT22, pero sigue siendo accesible por el botón 4 del mando IR —pulsadores y mando se diseñaron redundantes desde Act2—, por lo que la operativa no se degrada.

Mejoras futuras. Sustituir el potenciómetro por un módulo INA219 real manteniendo la misma API de `diagnostics`; añadir un *watchdog* hardware; y exportar la telemetría por Wi-Fi a un servicio externo, dando el salto a una instrumentación conectada propia del IoT industrial.

Bibliografía

- [1] A. Joachimsmeier, K. Shirriff et al. “Arduino-IRremote.” v4.x, compatible con ESP32. (), dirección: <https://github.com/Arduino-IRremote/Arduino-IRremote> (visitado 21-05-2026).

- [2] Espressif Systems. “ESP32 Technical Reference Manual.” ADC SAR, periférico LEDC, pines de *strapping*. (), dirección: https://www.espressif.com/sites/default/files/documentation/esp32_technical_reference_manual_en.pdf (visitado 21-05-2026).
- [3] Aosong Electronics. “AM2302/DHT22 Temperature and Humidity Sensor — Datasheet.” Rango medida: T $-40..80^{\circ}\text{C}$, H 0 % a 100 %. (), dirección: <https://www.sparkfun.com/datasheets/Sensors/Temperature/DHT22.pdf> (visitado 21-05-2026).
- [4] L. G. Corona Ramírez, G. S. Abarca Jiménez y J. Mares Carreño, *Sensores y Actuadores: Aplicaciones con Arduino*. México: Patria, 2014, Curva R-lux del LDR, p. 127; promediado de medidas, tema 6.
- [5] “Wokwi Documentation,” Wokwi. (), dirección: <https://docs.wokwi.com/> (visitado 21-05-2026).